

DeviceCluster — Developer Handoff

Everything needed to call the device-clustering pipeline from a UI.

What's in this bundle

Folder	Contents
lib/	Logic.dll, Model.dll + 22 dependency DLLs (Release build, .NET 10)
python/	The two prediction scripts and their trained model files (~8.5 MB)
sql/	Three scripts that create the required SQL Server tables
requirements.txt	Python package versions the models were trained against

Prerequisites

- **.NET 10 runtime, Windows x64.** Model.dll is built with RuntimeIdentifier=win-x64 and SelfContained=false, so the runtime must be installed on the machine — it isn't embedded. This is a Windows-only library.
- **SQL Server** access to the XenCreator database on Neptune (128.100.20.33).
- **One of two prediction sources** — see *Choosing a prediction client* below:
 - *Recommended* — network access to the **ML service** at http://128.100.8.213:8000 (SSSBPD01). Nothing to install: no Python, no packages, no model files.
 - *Or* — **Python 3.13** with pip install -r requirements.txt, plus the python/ folder from this bundle. The .pkl files are version-sensitive; scikit-learn in particular will warn or fail to unpickle across major versions, so use the pinned versions.

Setup

1. Create the tables — run each script in sql/ against XenCreator. All three are IF NOT EXISTS-guarded, so re-running them is safe.

Script	Table	Role
PatternCluster.sql	dbo.PatternCluster	Input — quota patterns per customer
DeviceReviewQueue.sql	dbo.DeviceReviewQueue	Output — devices needing review
OutputDeviceAssignment.sql	dbo.OutputDeviceAssignment	Output — successfully assigned devices

The source table (dbo.DummyTestingData by default) is expected to already exist.

2. Only if you are running predictions locally — keep the `python/` folder structure intact. Both scripts resolve their model paths relative to their own file location, so `predict_equipment.py` must stay next to `predict_equipment_folder/`, and likewise for `predict_sectioncluster.py`. Moving a script away from its folder breaks it. If you use the ML service you can ignore the `python/` folder entirely.

3. Reference `lib/Logic.dll` from your project. It pulls in `Model.dll` automatically. Keep the whole `lib/` folder together — including `runtimes/`, which holds the native SQL Server networking library. Without it, connections fail at run time with an error that does not name the cause.

Calling the pipeline

One method runs the whole thing. It never writes to the console and never prompts — safe to call from a UI thread, a service, or a scheduled job.

```
using Logic;
using Model.Services;

var sqlReader = new PythonSQL(connectionString);
var logic     = new LogicAssignment(connectionString);

// Where predictions come from. See "Choosing a prediction client" below.
IPredictionClient client = new HttpPredictionClient("http://128.100.8.213:8000");

var result = await DevicePipeline.RunAsync(
    sqlReader:      sqlReader,
    client:          client,
    logic:           logic,
    sqlSourceTable:  "dbo.DummyTestingData",
    sqlQuotaTable:   "dbo.PatternCluster",
    sqlOutputDir:    @"...\data",           // scratch folder for intermediate JSON
    projectCode:     "A9998",
    callbacks:       null);                 // optional – see below
```

Choosing a prediction client

`RunAsync` takes an `IPredictionClient`. Two implementations ship in `Model.dll`, and everything else about the call is identical either way.

```
// Predictions from the ML service – nothing to install locally.
IPredictionClient client = new HttpPredictionClient("http://128.100.8.213:8000");

// Or predictions from local Python – needs Python 3.13, the packages,
// and the python/ folder from this bundle on every machine that runs it.
IPredictionClient client = new PythonClient(
    pythonExe:        "python",
    scriptDeviceType: @"...\python\predict_equipment.py",
    scriptSectionCluster: @"...\python\predict_sectioncluster.py");
```

Prefer `HttpPredictionClient`. The service holds the models in memory, so a call costs a request rather than an interpreter start plus a model load, and no client machine needs Python or the `.pkl` files. Use `PythonClient` only where the service is unreachable.

`HttpPredictionClient` is disposable; if you construct it yourself, dispose it, or pass in an `HttpClient` you already own and manage.

What you get back

`DevicePipelineResult` carries every stage's output:

Property	Contents
<code>ClusterGroups</code>	Assigned devices grouped by section + cluster — the main result to render
<code>AssignedDevices</code>	Flat list of successfully placed devices
<code>UnknownPredictionDevices</code>	Floating — the model couldn't classify them
<code>UnallocatedDevices</code>	Floating — classified fine, but no quota room
<code>AllocationResult</code>	<code>VacancyReport</code> and <code>InitialDeficits</code> for quota reporting
<code>PipelineResults</code>	Raw per-device predictions, including ranked <code>TOP_CLUSTERS</code> candidates

Progress reporting

Pass a `DevicePipelineCallbacks` instance to drive a progress bar or live log. Every hook is optional — supply only the ones you need.

```
var callbacks = new DevicePipelineCallbacks
{
    OnProjectLoaded          = (req, json) => { /* devices loaded from SQL */ },
    OnDeviceTypesPredicted  = (results, secs) => { /* step 2 done */ },
    OnSectionsPredicted     = (results, lookup, secs) => { /* step 3 done */ },
    OnQuotaAllocated        = (quotas, alloc) => { /* allocation done */ },
    OnFloatingSplit         = (all, unknown, unallocated) => { /* review queue */ },
    OnClusterGroupsBuilt    = (assigned, groups) => { /* final result */ },
};
```

Where results are persisted

`RunAsync` writes both outcomes to SQL automatically — you don't need to persist anything yourself:

- Assigned devices → `dbo.OutputDeviceAssignment`
- Floating devices → `dbo.DeviceReviewQueue` (with a `Category` column distinguishing the two causes)

Both are upserted by `MERGE` on `DeviceId` + `ProjectCode`, so **re-running the same project is safe** — rows update in place rather than duplicating. A device that changes outcome between runs is removed from the

table it left, so it never appears in both at once.

To query results, read those two tables directly. `Status` in `dbo.DeviceReviewQueue` is `'pending'` until a device is manually placed.

Configuration

Table names are constructor/parameter arguments, not hardcoded — override them if your environment uses different names. The console app reads these environment variables as a convenience, but a UI can simply pass the values in directly:

Variable	Default
<code>SQL_SOURCE_TABLE</code>	<code>DummyTestingData</code>
<code>SQL_QUOTA_TABLE</code>	<code>dbo.PatternCluster</code>
<code>SQL_REVIEW_QUEUE_TABLE</code>	<code>dbo.DeviceReviewQueue</code>
<code>SQL_ASSIGNMENT_TABLE</code>	<code>dbo.OutputDeviceAssignment</code>
<code>PYTHON_EXE</code>	<code>python</code>

The connection string is supplied by the caller — the library never reads it from a registry or config file of its own.

Manual correction (optional)

`DevicePipeline.RunAsync` deliberately excludes the human-in-the-loop step. To let a user place a floating device by hand, call these on `LogicAssignment` after the run:

- `SuggestTopClusters(deviceId, knownDevices)` — ranked suggestions to show the user
- `AssignByNumericSimilarity(entry, knownDevices)` — resolve an UNKNOWN section/cluster
- `PlaceDevice(device, clusterGroups)` — place it, displacing a weaker device if needed
- `MarkAsAssigned(deviceId, projectCode, section, cluster)` — set `Status='assigned'` in SQL

Known limitations

- Windows x64 only, by build configuration.
- The `.pk1` models are tied to the pinned package versions in `requirements.txt`.
- Quota patterns for `SECTION 2` are real; sections 1 and 3–8 are placeholder data pending real numbers.
- No automated test suite — verification to date has been live runs against real SQL Server and trained models.
- **Device IDs are compared case-sensitively on the C# side.** Python normalises them, but `ClusterQuotaAllocator` (`HashSet<string>`) and `LogicAssignment` (`GroupBy/ToDictionary`) use the default ordinal comparer, so `V001.21` and `v001.21` are treated as two different devices. Same for `ProjectCode`, `Section` and `Cluster`. Worth fixing before the interface lets users type IDs by hand.

- **A tag prefix maps to exactly one device type, globally.** `extract_prefix_id` keeps only the leading letters, so `HT990` and `HT3137` are both `HT`. A manual correction overwrites that prefix for every device, past and future — the newest assignment wins. If a customer changes naming convention while old projects are still live, both meanings cannot coexist. The per-customer maps needed to scope this are already loaded (`customer_specific_map`) but are not used by the lookup.
- The ML service is registered as a Windows service and set to start at boot, but the server has not been restarted since, so boot-survival is configured rather than proven.